

Capstone Software Requirement Specifications

Version 1
Group 22
COMPSCI 4ZP6

Dr. Mehdi Moradi
October 10, 2025

Andre Menezes	meneza3@mcmaster.ca	400401293
Leiah Nay	nayl@mcmaster.ca	400362166
Martin Matlok	matlokm@mcmaster.ca	400369490
Nicholas Poulidis	poulidin@mcmaster.ca	400380146
Tony Lin	lint50@mcmaster.ca	400388309

Table of Contents

Document Information.....	4
Contribution History.....	4
Revision History.....	4
Glossary.....	4
1. Purpose of the Project.....	5
1.1 Background.....	5
1.2 Problem Statement.....	5
1.3 Problem Objectives.....	5
1.4 Proposed Solution.....	5
2. Stakeholders.....	6
2.1 Primary Stakeholders.....	6
2.2 Secondary Stakeholders.....	7
3. Mandated Constraints.....	7
3.1 Environmental Constraints.....	7
3.2 Schedule Constraints.....	7
3.3 Budget Constraints.....	7
3.4 Off-the-Shelf Services.....	7
4. Functional Requirements.....	8
4.1 Priority 0 - Minimum Viable Product.....	8
4.2 Priority 1 - Next Feature Set.....	9
4.3 Priority 2 - Non-critical Features.....	9
4.4 Priority 3 - Future Application Features.....	9
5. Data and Metrics.....	9
5.1 Profile Data.....	9
5.1.1 Skills.....	9
5.2 Projects.....	10
5.3 Activity.....	10
5.4 Metadata.....	10
5.5 Algorithm Metrics.....	10
6. Non-Functional Requirements.....	11
6.1 Look and Feel Requirements.....	11
6.2 Usability and Humanity Requirements.....	11
6.3 Performance Requirements.....	11
6.4 Operational and Environmental Requirements.....	11
6.5 Maintainability and Support Requirements.....	11
6.6 Security Requirements.....	12
6.7 Cultural Requirements.....	12
6.8 Compliance Requirements.....	12

7. Risks and Issues Predicted.....	12
7.1 User Adoption.....	12
7.2 Matching Algorithm Evaluation.....	12
7.3 Lack of Front-End Expertise.....	13
8. Team Meeting and Communication Plan.....	13
9. Team Member Roles.....	13
10. Workflow Plan.....	14
10.1 Version Control.....	14
10.2 Agile Methods.....	14
10.3 Storing Data.....	15
10.4 Meeting Requirements and Performance Metrics.....	15
11. Proof of Concept Demonstration Plan.....	15
12. Technology.....	15
12.1 Front-End.....	15
12.2 Back-End.....	16
12.3 Infrastructure.....	16
12.4 Development Workflow Tools.....	16
12.5 Testing.....	16
13. Project Scheduling.....	16

Document Information

Contribution History

Authors	Sections
Menezes	5.1, 5.1.1, 5.2, 5.3, 5.4, 12.1, 12.2, 12.3, 12.4, 12.5
Nay	4.1, 4.2, 4.3, 4.4, 5.5, 9, 11, 13
Matlok	2.1, 2.2, 3.4, 5.5, 7.1, 7.2, 7.3
Poulidis	1.1, 1.2, 1.3, 1.4, 3.1, 3.2, 3.3, 10.1, 10.2, 10.3, 10.4
Lin	3.4, 6.1, 6.2, 6.3, 6.4, 6.5, 6.6, 6.7, 6.8, 8

Revision History

Version	Authors	Description	Date
0	Menezes, Nay, Matlok, Poulidis, Lin	Initial Document	2025-09-30
1	Menezes, Nay, Matlok, Poulidis, Lin	Updated 1.4, 4, 5.5, 11 according to TA Feedback	2025-11-10

Glossary

FE	Front-end
BE	Back-end
AWS	Amazon Web Services
UI	User Interface
QA	Quality Assurance
HTTPS	Hypertext Transfer Protocol Secure
API	Application Programming Interface
IDE	Integrated Development Environment
CPU	Central Processing Unit
GPU	Graphics Processing Unit

REST	Representational State Transfer
SQL	Structured Query Language
NLP	Natural Language Processing

1. Purpose of the Project

1.1 Background

Students and early professionals often seek opportunities to collaborate on projects outside of their coursework or workplace. These projects help them strengthen their skills, build portfolios, and connect with like-minded individuals. However, finding reliable teammates or opportunities is often limited to personal networks, scattered online forums, and LinkedIn, making collaboration difficult.

1.2 Problem Statement

There is currently no dedicated, informal, and accessible platform designed for students and early professionals to discover both projects and collaborators easily. Without a central hub, individuals struggle to match their skills and interests with the right opportunities, leading to missed chances for learning, innovation, and community building.

1.3 Problem Objectives

The objectives of the project are to:

- Provide a streamlined way for users to connect with potential teammates or projects.
- Support project discovery through a modern, intuitive, like-dislike style interface.
- Enable users to create and customize profiles, showcasing their skills and goals.
- Encourage collaboration and networking among early-career professionals.

1.4 Proposed Solution

The solution is Peer.io, a mobile-based application where users can browse two interactive feeds: one for project ideas and one for potential collaborators. Using a like-dislike style interface, users can match with projects and peers. Once a mutual interest is established, contact information is exchanged to facilitate collaboration. Over time, the platform's matching algorithm will adapt to user preferences in combination with the user's skills and experience, making it easier to find meaningful and relevant opportunities.

Upon account creation, a user will be prompted to upload their resume. The system will automatically parse it using NLP to extract key information about the user, such as job titles, skills, projects,

experience descriptions, and technologies used. These extracted elements populate structured backend fields that represent the user's profile. Both the user's profile and a project description are then converted into vector embeddings that capture their semantic meaning. The system calculates a similarity score between these two vectors (using techniques like cosine similarity), along with additional factors such as skill overlap, keyword relevance, and experience recency. This score determines how closely the user's experiences align with the project's requirements. The users with the highest similarity scores are ranked as the best matches for that project, and the same approach can be applied in reverse to recommend the most relevant projects for each user. The matching feed display order will then be in decreasing order, where users will see the most relevant projects first and vice versa.

In the future (post proof of concept), this process will evolve so that the resume is parsed once to create a persistent, structured user profile (skills, roles, projects, etc), and the similarity score will also be calculated based on the projects users will add to their display page. Each added project is normalized, mapped to canonical skills, and embedded as vectors that become part of the user profile. In the matching process, the system will calculate a hybrid similarity score (resume + projects). Similarity scores will also update immediately when the profile changes, so newly added or edited projects directly affect the match quality shown to both users and project owners. On top of similarity score, an ELO system will also have weight in determining the order of users shown to projects to ensure higher quality candidates are assigned to higher quality / more desirable projects.

2. Stakeholders

2.1 Primary Stakeholders

- **Students / Early Professionals**
 - With limited to no work experience, students and early professionals often rely on projects to strengthen their resumes or hone their skills. Students will use Peer.io to seek collaborators for their own project ideas and to join existing projects.
- **Entrepreneurs / Startup Founders**
 - Small projects often struggle to attract motivated workers. Peer.io provides a platform for connecting with peers who may be more willing to take risks and collaborate on innovative ventures.
- **Freelancers**
 - Freelancers with technical or creative skills can leverage Peer.io to discover projects that expand their skillset, allowing them to improve their portfolio.
- **Course Stakeholders**
 - Professor Mehdi Moradi
 - Amirhossein Sabour
 - Sahib Khokha

2.2 Secondary Stakeholders

- **Users of Finished Projects**
 - As more projects are launched through Peer.io, the general public benefits indirectly from increased access to innovative solutions and products.
- **Employers**
 - With more opportunities for candidates to hone their skills and gain experience, future employers will be able to find more skilled candidates.

3. Mandated Constraints

3.1 Environmental Constraints

- Requires a constant internet connection.
- Requires an iPhone 11 or newer.
- Requires iOS 18 or newer.

3.2 Schedule Constraints

- A working proof of concept by November 21, 2025
- The final version is to be submitted for course evaluation on April 4, 2026.

3.3 Budget Constraints

- The project will rely on low-cost hosting solutions, with an estimated budget of $\leq$ \$25/month for server upkeep during development and demonstration.
- Free-tier tools and frameworks, such as React Native, Firebase, and Supabase will be prioritized.

3.4 Off-the-Shelf Services

- **LinkedIn**
 - Despite being a networking service, LinkedIn does not have a section directly dedicated to finding collaborators for projects. The purpose of Peer.io is to create a platform limited in scope to project collaboration.
- **Indeed**
 - Indeed is a well-established job search platform centred around connecting employers with candidates in search of part-time or full-time employment opportunities. Indeed is not designed for informal, unpaid project-based collaboration in mind. Peer.io is aimed at filling the space for early-career professionals to find smaller projects emphasizing career development rather than traditional job placement.

4. Functional Requirements

4.1 Priority 0 - Minimum Viable Product

- Front-End Requirements: App Development
 - Project browsing feed to look through the various projects available for collaboration
 - People browsing feed to look through people who want to collaborate
 - Must have at least one project on your profile to look for people
 - Account sign-up and sign-in pages
 - Project creation page to post a short description of your project idea
 - Post and profile interactions
 - Liking and disliking profiles or projects
 - If a user likes your project, and you like their profile, you will be forwarded their contact information to begin collaborating
 - Likes page to see who has liked your profile or specific project
 - Matches page to see the people and projects you have matched with, and their contact information
 - Resume parsing page for users to upload their resume
- Matching Algorithm:
 - Matching projects/profiles based on the highest relevance score using an NLP-based semantic similarity, with explainable scoring and tunable weights
 - Parse a resume to analyze job and project descriptions to populate the user profile.
 - Allow the user to upload any additional projects or skills to their profile
 - Utilize an NLP algorithm to analyze similarity and compute scores and a ranked list of the most similar projects/profiles.
 - Display the top-matched projects/profiles on respective feeds.
 - Elo matching system for profiles and projects
 - Elo scores will increase or decrease based on interactions with a user's profile or project (i.e. the more likes a profile gets, the higher the elo score)
 - We will be using a project optimal matching system
- Resume Parsing:
 - User can upload their resume which will be parsed to fill in their initial profile
 - Preexisting NLP software will be used to identify skills and skill level
- Database Requirements:
 - Database setup to store project and user profile data
 - Profiles include name, profile pictures, location, mentor/mentee option, education, a brief portfolio with links and descriptions, work experience, interests, skills and skill level (decided by the user on a five-point scale)
 - Projects include name, group size, description, skills needed with minimum skill level, project intention (school project, resume builder, start up), topic(s) of interest, and location

- APIs for the client-side application to retrieve and edit their profiles and projects
- APIs for the server to access profiles and projects
- The client will reach out to the server for profile/project recommendations, the server will compute these recommendations using the matching algorithm and available profiles/projects from the database.

4.2 Priority 1 - Next Feature Set

- Filtering on projects and people
 - Capability to filter candidate profiles based on a specific project
 - Ability to filter based on skill level, skills, and interests
- Profile and project customization
- Messaging system to communicate after matching
- Allow multiple projects per person
- Ability to delete or pause (temporarily remove from matching feeds) your project or profile

4.3 Priority 2 - Non-critical Features

- Tag candidates with which of your projects they would best fit

4.4 Priority 3 - Future Application Features

- Upload the application to the App Store
- Classroom feature
 - Limit the people and projects someone can browse to within the classroom
 - Same functionality, limit the scope
- Incorporating previous matches, posts, inactivity and more into the matching algorithm
- Upload your resume to auto-fill your profile
- Ability to add a project to your archive, a set of projects a user has previously worked on and found through Peer.io

5. Data and Metrics

5.1 Profile Data

- Data Types: Mostly text (skills, interests, bios), images (profile photos), and dates (availability).
- Purpose: Helps determine which projects a person is best suited for.
- Metric Example: Percentage of profiles with complete skills/interests (data quality).

5.1.1 Skills

- This will be the primary data used to match people to projects. Each profile includes a list of skills (e.g., “Python,” “UI Design,” “Project Management”). When matching, these skills are

compared against the project's required skills. There will also be a skill level, measured in a numerical scale, for each skill.

1. Exact overlap → Direct skill match (e.g., “Python” ↔ “Python”).
2. Related categories → Broader groupings (e.g., “UI/UX” ↔ “Figma, Wireframing”).
3. Weighted scoring → Profiles with more overlapping required skills rank higher in recommendations.

5.2 Projects

- Data Types: Text (title, description, requirements), numbers (team size), dates (start/end timelines), and some images (project photos/logos).
- Purpose: Defines opportunities that can be matched with profiles.
- Metric Example: Time-to-fill (days from project creation to match).

5.3 Activity

- Data Types: Event logs with text (types of actions like/swipe/apply), numeric counters (likes, matches), and timestamps.
- Purpose: Capture user preferences and behaviours to improve recommendations over time.
- Metric Example: Conversion rate (number of projects liked → matches).

5.4 Metadata

- Data Types: Text (collaboration preferences, match reasons), dates (project timelines, match history), and structured logs.
- Purpose: Provides context to smooth and personalize the experience.
- Metric Example: Match success rate (number of successful matches / total matches).

5.5 Algorithm Metrics

Since our matching algorithm does not have a unique or exact result, quantitative metrics will be difficult to measure. Our matching algorithm will use natural language processing to determine the similarity between a user's profile and a given project. The true accuracy of this algorithm will be difficult to determine without real-world use, as user data such as retention and match speed is unobtainable without a large, active user set. As a compromise, during testing, we will run on a synthetic set of users and projects where every user will have an expected list of projects ordered by compatibility. Then, we will evaluate our algorithm's accuracy based on whether the order of projects presented to a user is similar to the expected compatibility list we had determined prior.

6. Non-Functional Requirements

6.1 Look and Feel Requirements

- The interface should have a minimalist, clean design and consistent typography.
- Colours on the app should remain consistent with our logo.
- Layouts must remain uncluttered, even as new features (filters, tags, profiles) are added.
- Interaction patterns should feel familiar and consistent with popular apps (TikTok/Instagram/Hinge).

6.2 Usability and Humanity Requirements

- Infinite scrolling for feeds with smooth transitions.
- Core interactions (like, profile tap) should require two taps max. For example, one tap for a like button, or a double-tap anywhere on the screen for a like as well.
- Users should receive clear error messages. For example, if the server is down or project creation fails.
- Every possible action should be intuitive. For example, from the home page, how would we get to the profile section? This should be easily findable through meaningful icons/words.

6.3 Performance Requirements

- Loading new feed items should take ≤ 200 ms per scroll and should feel instantaneous.
- Switching between feeds/tabs should take ≤ 200 ms and should feel instantaneous.
- The system should support at least 100 concurrent users without noticeable degradation.
- Image compression must be used so project cards load quickly.
- Basic features, such as liking, should also feel instantaneous.
- Posting a project should take ≤ 1 second.
- Starting up the app should take ≤ 3 seconds.

6.4 Operational and Environmental Requirements

- The system will be deployed as a mobile iOS application, accessible on all iPhones and iOS versions currently supported by Apple. Older versions will not be supported.
- Must run on low-cost hosting.
- Requires a stable internet connection for all features (no offline mode).
- Should degrade gracefully if images or media fail to load, such as using placeholder icons/text.

6.5 Maintainability and Support Requirements

- The codebase should be modular, with a good, simple programming design.
- Implement the use of build tools for dependency management, build automations, and good project structure.
- Documentation must not be overlooked.

- The system should be easy to update with new features (such as filters, tags, and algorithms) without requiring major refactoring. This goes the other way around as well; taking out a feature should not crash the system.

6.6 Security Requirements

- Passwords must not be stored as plain text..
- Communication between client and server must be encrypted (HTTPS).
- User contact information should only be shared after mutual matching.
- Session management should include a timeout after inactivity.
- Prevent common attacks: input sanitization for all inputs and rate limiting for login attempts.
- Users can only perform actions on their own account. For example, a user can't create a project for someone else.
- API keys stored via edge functions (server-side).

6.7 Cultural Requirements

- The platform should be inclusive:
 - Support neutral, non-discriminatory language throughout the UI.
 - Avoid symbols or colour choices that might carry unintended cultural meanings. For example, red/green for accessibility, not for “good/bad person”.
- Designed to encourage collaboration and mutual respect among students/entrepreneurs.

6.8 Compliance Requirements

- Must comply with basic privacy regulations.
- Store only necessary user data (minimization principle).
- Include terms of service specifying how we handle data and more.

7. Risks and Issues Predicted

7.1 User Adoption

- Without an established user base, there is a risk of limited project listings, which may discourage new users from joining. To ensure sustainability beyond the capstone project, the team will need to develop a clear adoption and launch strategy, with a strong focus on user acquisition and community growth.

7.2 Matching Algorithm Evaluation

- As described in 7.1, there will not be a large amount of data regarding projects until a user base is established. This will result in difficulties evaluating the quality of our matching algorithm,

as without a large number of projects and users, the intricacies of the matching algorithm will not be as visible.

- Evaluating the success of the algorithm will also be very difficult without monitoring results from app usage over time. An algorithm like this will likely need to be tuned over time after viewing results from usage, but without a user base, we will have to make a strong attempt and hope it is successful.

7.3 Lack of Front-End Expertise

- The team currently lacks a plethora of experience in overall front-end design and development. This gap will be mitigated through active learning efforts from the whole team, led by Nicholas and Andre.
- The team will aim to gain proficiency in front-end work for future career endeavours, on top of providing that skill set to the project.

8. Team Meeting and Communication Plan

We will typically have weekly group meetings in person somewhere on campus, an example, at Thode. When someone cannot make it to a specific meeting in person, we will either reschedule for another day or we will have our meetings online on FaceTime or Discord. Typically, we would not have a group meeting unless everyone is available either for an in-person meeting or an online meeting.

We will communicate with each other through iMessage or Discord. We will also use GitHub Projects for project management: tracking all the features/milestones, assigning priorities to each, and assigning tasks to people, as well as assigning deadlines for said tasks.

9. Team Member Roles

- **Andre Menezes:** Algorithm Lead, Full Stack Developer
 - Lead development of the matching algorithm.
 - Assist with the development of the application.
- **Leiah Nay:** Cybersecurity Lead, UI Designer
 - Ensuring proper sanitization of all inputs and safe storage of user information.
 - Designing the layout and interactions of the application.
- **Martin Matlok:** Project Manager, QA Lead
 - Take meeting notes and organize workflow to ensure the project is delivered in a timely and efficient manner.
 - Design tests for all aspects of the application to ensure a bug-free user experience.
- **Nicholas Poulidis:** Front-End Lead
 - Lead development of the application's front-end user interface.
- **Tony Lin:** Back-End Lead
 - Ensuring information is stored and retrieved efficiently and properly.

Menezes, Nay, Matlok, Poulidis, Lin

- Lead integrations from front-end to back-end.

Everyone will also function as a full-stack developer as the need arises.

10. Workflow Plan

10.1 Version Control

We will be using GitHub for version control and pull requests. Work will be done on branches to prevent overwriting our main branch. Every pull request will require approval from at least one other group member before merging onto the main branch. In addition to an approval, pull requests will require a short description containing information on the changes made and links to any related issues. The reviewer should also leave comments to capture any changes they believe should be made before merging. Additionally, all changes should be made on a feature branch, then merged into main with the proper review requirements met.

10.2 Agile Methods

We will use a shortened version of Agile with small sprints to help us plan out work and ensure we are readily meeting our schedule constraints. For capturing our work, we will be using GitHub projects, which will provide a kanban board, backlog, and roadmap. Branches created for feature implementation or bug fixes will be linked to an issue that can be seen on the kanban board. This will ensure we understand the work that has been done during a given sprint, work that comes up that we did not originally plan, and what work remains. For work planning, we will leverage weekly “sprints” where each member will be designated issues from our backlog to work on for the week. This work should be able to be completed before the next weekly sprint planning session. Tasks that are predicted to take longer than a week will be broken down into smaller issues. To estimate how much work each task might take, we will utilize story points. One story point is an easy task that will take a day or less, two story points are a multi-day task, three story points being a week-long task, and five story points to indicate a task to be broken down into smaller sub-tasks.

These agile methods will not be hard-set and can be moved past if we find the work is being blocked due to unforeseen challenges or dependencies. The goal of this framework is to guide our progress, not restrict it. If certain tasks require additional time or resources, we will adapt our sprint goals accordingly while maintaining transparency through updated issues and sprint reviews.

Sprint planning sessions will be held every Monday, with the first portion of our meeting going over a review of our previous sprint.

10.3 Storing Data

Our data will be stored primarily using Supabase, but we will leverage AWS storage systems for any additional storage needs.

10.4 Meeting Requirements and Performance Metrics

To meet the schedule constraints mentioned, we will leverage the aforementioned Agile methods to ensure a timely delivery. Performance metrics will be manually tested in test phases to ensure they are being met. For security requirements, we will leverage the security and control of AWS and Supabase, as well as ensure that work captured on GitHub is kept private. We will also leverage appropriate Python and JavaScript/Typescript security libraries.

11. Proof of Concept Demonstration Plan

There are two big risks we foresee. Firstly is our lack of front-end and database experience as a team. The second risk will be developing an adequate matching algorithm. While these are risks, it is trivial that solutions to these problems exist according to existing applications.

For our proof of concept, we would like to have a basic mobile application with some of the pages described above. We will not yet have fully displayed candidate or project profiles; however, the browsing feeds should still be able to iterate through the database of profiles and projects. We will more importantly have the first iteration of our matching algorithm completed. The elo matching will not yet be incorporated, so similarity scores between projects and profiles will determine matches. These similarity scores will be determined with [preexisting NLP software](#) that uses a resume and a project description as input. The provided software will be used in our first iteration but other options will be explored during the research period as detailed in the Gantt Chart below.

Our demonstration will consist of a user creating an account, uploading their resume and having that parsed to automatically fill in their profile, creating a project and then browsing through some projects and candidates. Then we will simulate the matching process by liking a candidate and having a candidate like our project. Finally, we will show the matched feed, where all candidates who have liked your project will be displayed.

12. Technology

12.1 Front-End

We will use TypeScript as our front-end language. The framework of choice will be **React Native (with Expo)**, which enables fast development cycles. As per state management, we will be using **Redux Toolkit**, and styling will be handled with **Tailwind via NativeWind** for speed, responsiveness,

Menezes, Nay, Matlok, Poulidis, Lin

and maintainability. Supporting tools will include **Visual Studio Code** as the IDE with extensions such as **ESLint**, **Prettier**, and **Black** for linting and formatting. Version control will be managed through **GitHub**.

12.2 Back-End

We will be using **Python** as the back-end language, with **FastAPI** as the framework, due to its compatibility with future machine learning integrations. Our primary database will be **Supabase** (Postgres-based), which will also provide authentication services. We will additionally integrate **Redis** for caching matches/swipes to reduce latency. **No GPU resources** are required, as the system primarily relies on rules, embeddings, and PostgreSQL, which are lightweight and CPU-friendly.

12.3 Infrastructure

The mobile app will connect to APIs hosted on AWS, with REST endpoints exposed by the back-end. Supabase will manage persistent storage and authentication.

12.4 Development Workflow Tools

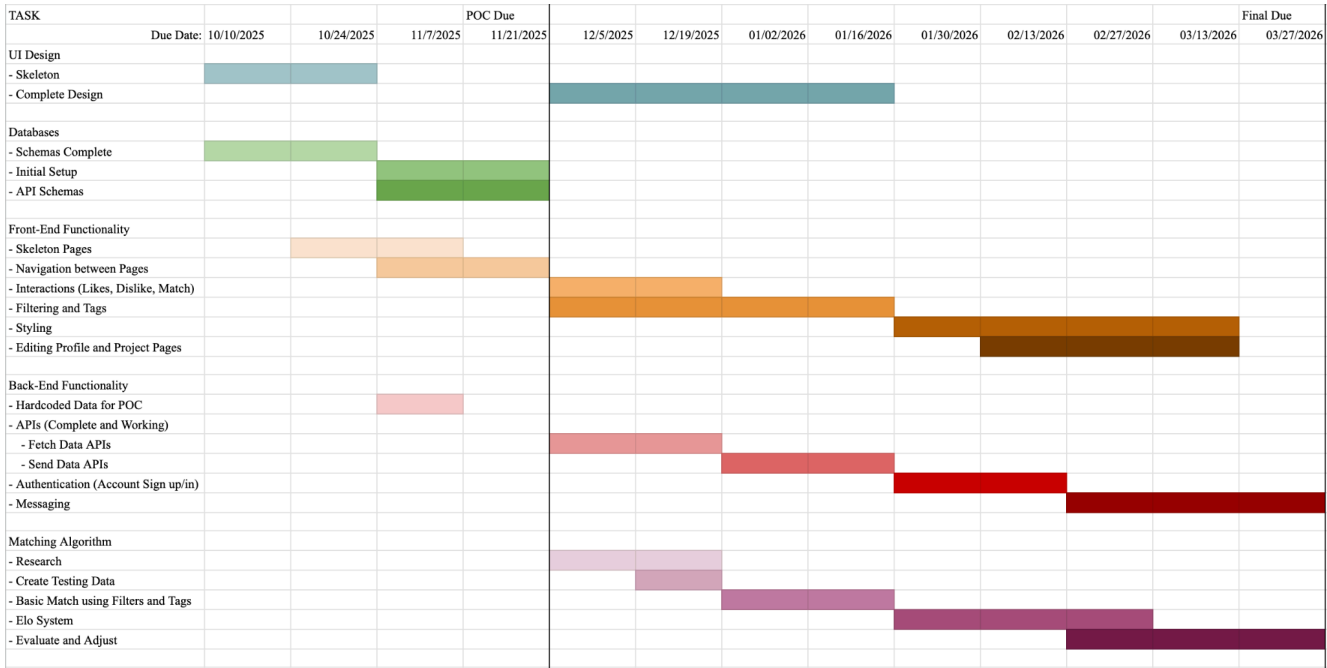
- Visual Studio Code (IDE with ESLint and Prettier extensions)
- Git (Version Control)
- GitHub (Hosted repository management)

12.5 Testing

- For back-end unit testing, PyTest will be used to create tests for all major features.
- If a bug is encountered, a unit test will be created once the issue is fixed to ensure no future changes reintroduce the issue.
- For front-end unit tests, Jest will be used.

13. Project Scheduling

The following is a Gantt Chart detailing our plan for progress throughout this project.



We have planned our schedule in two-week sprints. Testing, both manual and unit, will occur biweekly alongside the sprints. The details of this chart can also be seen below.

Task	Start Date	End Date
UI Design	10/10/2025	01/16/2026
• Skeleton	10/10/2025	10/24/2025
• Complete Design	11/21/2025	01/16/2026
Databases	10/10/2025	11/21/2025
• Schemas Complete	10/10/2025	10/24/2025
• Initial Setup	10/24/2025	11/21/2025
• API Schemas	10/24/2025	11/21/2025
Front-End Functionality	10/17/2025	03/13/2026
• Skeleton Pages	10/17/2025	11/7/2025
• Navigation between Pages	10/24/2025	11/21/2025
• Interactions (Likes, Dislikes, Match)	11/21/2025	12/19/2025
• Filtering and Tags	11/21/2025	01/16/2026
• Styling	01/16/2026	03/13/2026
• Editing Profile and Project Pages	01/30/2026	03/13/2026
Back-End Functionality	10/24/2025	03/27/2026

• Hardcoded Data for POC	10/24/2025	11/7/2025
• APIs (Complete and Working)	11/21/2025	01/16/2026
• Fetch Data APIs	11/21/2025	12/19/2025
• Send Data APIs	12/19/2025	01/16/2026
• Authentication (Account Sign up/in)	01/16/2026	02/13/2026
• Messaging	02/13/2026	03/27/2026
Matching Algorithm	11/21/2025	03/27/2026
• Research	11/21/2025	12/19/2025
• Create Testing Data	12/5/2025	12/19/2025
• Basic Match using Filters and Tags	12/19/2025	01/16/2026
• Elo System	01/16/2026	02/27/2026
• Evaluate and Adjust	02/13/2026	03/27/2026